

MOCK INTER IIT TECH MEET 15.0

IIT Ropar | Internal Preparatory Edition

FinAgent

Real-Time AI Investment Decision System

HIGH PREP | Problem Statement

Inspired by Pathway — Inter IIT Tech Meet 14.0

1. Background & Motivation

Financial markets generate a relentless stream of signals — price ticks, earnings calls, regulatory filings, macroeconomic releases, and news events — every second of the trading day. Yet most existing decision-support systems are batch-oriented, processing data in periodic snapshots that are stale the moment they are produced. Decisions that arrive even minutes late can mean missed opportunities or unmanaged risk exposure.

Recent breakthroughs in large language models (LLMs) and multi-agent architectures have opened a new frontier: systems that can reason over live data streams, retrieve contextually relevant information on demand, and coordinate specialised agents to execute complex analytical workflows — all with minimal human intervention. Frameworks such as LangChain and LangGraph have made it practical to compose such systems from modular, reusable components.

This problem statement challenges you to build exactly that: a production-grade, multi-agent AI system that ingests market signals in real time, retrieves relevant context from a live knowledge base using Retrieval-Augmented Generation (RAG), and produces structured investment decision artefacts — all through an orchestrated agentic pipeline built with LangChain and LangGraph.

2. Problem Statement

Design and implement a functional, multi-agent AI system — called FinAgent — that monitors a stream of market-related inputs, synthesises information from a retrieval-augmented knowledge base, and produces structured investment decision artefacts in near real time. The system must be built using LangChain for agent tooling and LangGraph for multi-agent orchestration.

2.1 Signal Ingestion & Preprocessing Agent

The system must include a dedicated agent responsible for ingesting and normalising incoming market signals. Supported input modalities are:

- Structured numerical data: stock price ticks, volume, and simple derived indicators (e.g., moving averages, RSI) from a simulated or real market data feed.
- Unstructured text: financial news headlines and short article summaries provided as a text stream (e.g., RSS feed or CSV batch).
- Semi-structured documents: earnings call transcripts or regulatory filings uploaded as PDF or plain-text files.

The agent must parse, timestamp, and normalise all inputs into a unified internal representation before passing them downstream.

2.2 RAG-Powered Context Retrieval Agent

The system must maintain a knowledge base built from a corpus of financial reference documents (e.g., uploaded earnings transcripts, sector reports, or macro-economic summaries). A dedicated retrieval agent must:

- Chunk and embed documents into a vector store upon ingestion.
- Accept natural-language queries from other agents and return the top-k most relevant passages.
- Ground retrieved passages with source attribution (document name, page or section reference).

The vector store and embedding model must be open-source (e.g., FAISS with Sentence Transformers, Chroma, or equivalent). Commercial embedding APIs are permitted as a secondary option but not as the sole solution.

2.3 Signal Analysis & Hypothesis Agent

A reasoning agent must analyse the normalised signals and retrieved context to:

- Classify each incoming event as bullish, bearish, or neutral with a brief rationale.
- Generate a structured investment hypothesis: a concise statement of the opportunity or risk, the supporting evidence, and the expected time horizon (intraday, short-term 1–5 days, medium-term 1–4 weeks).
- Assign a confidence score (0–100) to each hypothesis based on signal strength and evidence quality.

This agent must query the RAG retrieval agent to ground its reasoning in the knowledge base before finalising any hypothesis.

2.4 Decision Synthesis & Alerting Agent

A final orchestration agent must aggregate hypotheses from the analysis agent and produce structured decision artefacts. Each artefact must include:

- Asset or sector under consideration.
- Recommended action: BUY, SELL, HOLD, or WATCH (no action yet, monitor).
- Confidence level: High (>70), Medium (40–70), or Low (<40).
- Key supporting evidence: two to three bullet points drawn from signals and retrieved context.
- Risk flags: any contradictory signals or data gaps that reduce confidence.

Artefacts must be displayed in the system's user interface in real time as they are generated. The agent must also emit an alert (console log or UI notification) for any hypothesis with confidence above a configurable threshold.

3. Technical Scope & Constraints

3.1 Required Technology Stack

The following components are mandatory:

- LangChain: for individual agent construction, tool definitions, prompt templates, and LLM integration.
- LangGraph: for multi-agent orchestration, state management, and inter-agent message passing.
- Open-source vector store: FAISS, Chroma, Qdrant, or equivalent for the RAG knowledge base.
- Open-source embedding model: Sentence Transformers (e.g., all-MiniLM-L6-v2) or equivalent.
- LLM backbone: any open-source model (e.g., Llama 3, Mistral, Gemma) or a commercial API (OpenAI, Gemini, Claude). Teams must clearly declare which LLM they use.

3.2 Data Sources

Teams may use any combination of the following:

- Simulated market data: a CSV or JSON file of historical OHLCV (open, high, low, close, volume) data replayed as a stream.
- Real market data APIs: Yahoo Finance (yfinance), Alpha Vantage free tier, or similar free-tier APIs.
- News text: publicly available financial news RSS feeds or pre-downloaded headline datasets.
- Reference documents: any publicly available earnings transcripts, 10-K filings, or sector reports in PDF or text format.

Teams are responsible for ensuring their data sources comply with the respective terms of service.

3.3 Prohibited Components

The following are explicitly disallowed:

- End-to-end use of a single monolithic LLM call that bypasses the multi-agent architecture.
- Automated code generation or execution agents (custom coding agent functionality is out of scope).
- Any live trading integration, real brokerage API, or actual order placement.
- Proprietary data feeds requiring paid subscriptions as the primary data source.

3.4 Deliverables

1. A working application (locally runnable) with a web or terminal UI demonstrating the full pipeline.
2. Complete source code submitted as a compressed archive or public repository link.
3. A technical report (maximum 6 pages, double-column format) covering: system architecture, agent design, LangGraph state graph, RAG pipeline details, and sample output artefacts.
4. A 5-minute live demonstration during the final evaluation session.

4. Agent Architecture Requirements

The system must implement a clearly defined multi-agent architecture using LangGraph. The following requirements apply:

Agent	Primary Role	Key Interactions
Ingestion Agent	Parse and normalise market signals	Pushes normalised events to Graph state; triggers Analysis Agent
Retrieval Agent	Serve RAG queries from vector store	Receives queries from Analysis Agent; returns ranked passages with citations
Analysis Agent	Classify signals and generate hypotheses	Calls Retrieval Agent for context; passes hypotheses to Decision Agent
Decision Agent	Synthesise artefacts and emit alerts	Receives hypotheses from Analysis Agent; writes final artefacts to output

The LangGraph state graph must be explicitly documented (as a diagram or adjacency description in the technical report). Each agent must be implemented as a LangGraph node. State transitions between nodes must be governed by typed conditions (e.g., confidence threshold, event type) rather than unconditional edges where meaningful.

Agent communication must pass through the shared LangGraph state object. Direct function calls between agents that bypass the graph are not permitted.

5. Interface & Output Requirements

The system must provide a user-accessible interface that satisfies the following:

5.1 Knowledge Base Management

- An upload interface (file picker or folder path) for adding documents to the RAG knowledge base.
- A confirmation message indicating successful ingestion, chunk count, and embedding completion.
- A simple search interface to directly query the knowledge base and inspect retrieved passages (for debugging and evaluation).

5.2 Signal Feed

- A configurable input for specifying the market data source (file path, API ticker symbol, or simulated stream).
- A live feed panel displaying the most recent N signals as they are ingested, with timestamps.

5.3 Decision Artefact Display

- A structured card or panel for each generated artefact showing: asset, action, confidence level, evidence bullets, and risk flags.
- Artefacts displayed in reverse chronological order (newest first).
- Visual differentiation between BUY, SELL, HOLD, and WATCH recommendations (e.g., colour coding).
- Alert banner or notification for any high-confidence artefact (threshold configurable, default 70).

5.4 Observability (Required)

Agentic systems must be observable. The interface must expose, at minimum:

- An agent trace log showing which agent was active, what tools it called, and what it returned — per event processed.
- The retrieved RAG passages (with source attribution) that informed each hypothesis.
- Any error or fallback events, with a human-readable explanation.

5.5 Bonus Features (Not Required)

The following are rewarded under the innovation component of judging:

- Multi-turn conversational interface: users can ask follow-up questions about a specific artefact (e.g., 'Why did you rate this as high confidence?').
- Backtesting mode: replay a historical signal file and evaluate the accuracy of past hypotheses against subsequent price movements.
- Exportable report: download all artefacts and the agent trace log as a structured PDF or JSON file.

This is a simplified mock problem statement prepared by IIT Ropar for internal preparatory purposes. It is inspired by, but not identical to, the original Pathway problem statement from Inter IIT Tech Meet 14.0. [View Original Problem Statement](#)